Task 1 — Taxi-Level Trip Efficiency Analysis

COSC2637 Big Data Processing — Assignment 1 · Solution walkthrough (explanatory, not the submitted work log) · Verified working on the real EMR cluster (s4271512)

1. The problem, in plain terms

For every taxi, and for every "trip type" that taxi has driven (short/medium/long, based on distance), we need one summary row telling the transport authority: how many trips of that type, the biggest fare, the smallest fare, and the average fare. So a taxi that does a mix of short airport hops and long cross-city runs ends up with *two* output rows — one for its short-trip behaviour, one for its long-trip behaviour — rather than one row that blends both together.

Trip type is decided purely by distance:

Trip type	Distance condition
`long`	distance ≥ 200
`medium`	100 ≤ distance < 200
`short`	distance < 100

The "primary key" of the output is the pair `(taxi_id, trip_type)` — that's what every row is grouped and aggregated by.

2. The three files, and how they fit together

This is a classic single-pass Hadoop Streaming MapReduce job: one mapper stage, one reducer stage, no chaining. Three files make it up:

1. `task1_mapper.py` — reads raw trip rows, classifies them, and pre-aggregates ("in-mapper combining") before emitting anything.
2. `task1_reducer.py` — merges the pre-aggregated pieces from every mapper into the true final totals per (taxi_id, trip_type), and computes the average.
3. `task1-run.sh` — the driver: wipes any stale output, then launches the Hadoop Streaming job with the right configuration (3 reducers, the partitioner that keeps our 2-field key grouped correctly).

3. `task1_mapper.py`

```
#!/usr/bin/env python3
import sys

stats = {}  # (taxi_id, trip_type) -> [count, max_fare, min_fare, sum_fare]

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    fields = line.split(",")
    if len(fields) != 8:
        continue  # skip malformed lines

    taxi_id, fare, distance = fields[1], fields[2], fields[3]
    try:
        fare = float(fare)
        distance = float(distance)
    except ValueError:
        continue

    if distance >= 200:
        trip_type = "long"
    elif distance >= 100:
        trip_type = "medium"
    else:
        trip_type = "short"

    key = (taxi_id, trip_type)
    if key in stats:
        s = stats[key]
        s[0] += 1
        if fare > s[1]: s[1] = fare
        if fare < s[2]: s[2] = fare
        s[3] += fare
    else:
        stats[key] = [1, fare, fare, fare]  # count, max, min, sum

for (taxi_id, trip_type), (count, max_fare, min_fare, sum_fare) in stats.items():
    print('%s\t%s\t%d\t%.2f\t%.2f\t%.2f' % (taxi_id, trip_type, count, max_fare, min_fare, sum_fare))
```

What it does, step by step

- **Hadoop hands this script raw lines on stdin.** In Hadoop Streaming, the mapper is just an executable that reads text from standard input and writes text to standard output — no special API. That's why it's `for line in sys.stdin`, not some Hadoop-specific call.
- **Parsing:** each line of `Trips.txt` looks like `0,470,117.32,64.27,54.736,91.185,20.488,93.348` — comma separated: trip_id, taxi_id, fare, distance, pickup_x, pickup_y, dropoff_x, dropoff_y. We only need `fields[1]` (taxi_id), `fields[2]` (fare), `fields[3]` (distance) for this task, so the rest are read but ignored.
- **Defensive parsing:** `if len(fields) != 8: continue` and the `try/except ValueError` around the float conversions mean a malformed line is silently skipped rather than crashing the whole mapper task (which would fail the entire job).
- **Classification:** a simple if/elif/else turns the numeric distance into one of the three trip-type strings.
- **The important part — in-mapper combining:** instead of printing a line *per trip* (which the assignment explicitly discourages — "a reasonable big data program should not… heavy processing should be pushed to the mapper"), we

keep a Python dict `stats` keyed by `(taxi_id, trip_type)`. Every trip just updates that key's running count/max/min/sum in memory. Nothing is printed inside the loop at all.

- **Emit only at the end:** once `sys.stdin` is exhausted (this mapper task has no more input to read), the final loop prints *one line per distinct key this mapper ever saw* — not one line per trip. If a mapper processed 5,000 trips but they only belonged to 300 distinct (taxi, type) combinations, it emits 300 lines, not 5,000. That's the entire point of in-mapper combining: it shrinks the data volume that has to be shuffled across the network to the reducers.

> **Why is this "state preserved across input lines"?** Hadoop Streaming launches your mapper script *once per input split* and feeds it many lines over stdin in sequence — the process stays alive for the whole split. Because `stats` is declared once, outside the loop, it accumulates across every line that specific mapper process sees, rather than being reset per line. That persistence is exactly what the assignment means by "in-mapper combining with state preserved across input lines."

What the mapper emits

Tab-separated, 6 fields — but note these are still *partial* aggregates (one mapper only saw a slice of the whole file), not the final answer yet:

```
taxi_id   trip_type   partial_count   partial_max_fare   partial_min_fare   partial_sum_fare
```

4. `task1_reducer.py`

```
#!/usr/bin/env python3
import sys

current_taxi = None
current_type = None
count = 0
max_fare = None
min_fare = None
sum_fare = 0.0

def emit(taxi_id, trip_type, count, max_fare, min_fare, sum_fare):
    avg_fare = sum_fare / count
    print('%s\t%s\t%d\t%.2f\t%.2f\t%.2f' % (taxi_id, trip_type, count, max_fare, min_fare, avg_fare))

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    taxi_id, trip_type, c, mx, mn, sm = line.split('\t')
    c = int(c); mx = float(mx); mn = float(mn); sm = float(sm)

    if taxi_id == current_taxi and trip_type == current_type:
        count += c
        sum_fare += sm
        if mx > max_fare: max_fare = mx
        if mn < min_fare: min_fare = mn
    else:
        if current_taxi is not None:
            emit(current_taxi, current_type, count, max_fare, min_fare, sum_fare)
        current_taxi, current_type = taxi_id, trip_type
        count, max_fare, min_fare, sum_fare = c, mx, mn, sm

if current_taxi is not None:
    emit(current_taxi, current_type, count, max_fare, min_fare, sum_fare)
```

What it does, step by step

- **Hadoop guarantees sorted, grouped input.** Before the reducer ever runs, Hadoop's shuffle-and-sort phase collects every mapper's output for a given key and delivers it to one reducer, *sorted by key*. So by the time this script reads stdin, all the lines for taxi "104"/"long" are guaranteed to arrive consecutively — never interleaved with a different key in between. That's what makes the "streaming group-by" pattern below correct.
- **The running-group pattern:** this is the classic Hadoop Streaming reducer idiom (the same shape you'll have seen in the Week 3/4 word-count and salary examples). We track `current_taxi`/`current_type` — "which key am I currently accumulating for?" — plus the running totals for that key.
- **While the key stays the same:** merge this line's partial numbers into the running totals — add counts, add sums, take the max of maxes, the min of mins. This is combining *already-aggregated* partial results from potentially several different mappers, not re-deriving anything from scratch. That satisfies "reducers should mainly focus on final aggregation of already partially processed data."
- **The moment the key changes:** the previous key is finished — nothing more for it will ever arrive (thanks to sorted input) — so we call `emit()` to print its final line, then reset the running totals to start accumulating the new key.
- **The average is computed here, and only here:** `sum_fare / count` inside `emit()`. Averages can't be combined losslessly (the average of two averages isn't the true average unless you also weight by count) — that's exactly why the

mapper/earlier reducer stages ship the raw *sum* and *count* separately, and division only happens once, at the very last step.

- **The final `if current_taxi is not None: emit(...)` after the loop:** without this, the very last group read from stdin would never get printed — the loop only emits when it detects a key change, but there's no "next different key" event after the last line to trigger that. This line is easy to forget and is a classic off-by-one bug in this exact pattern.

What it emits — the actual final output

```
taxi_id   trip_type   total_trips   max_fare   min_fare   avg_fare
```

Real verified line from the cluster run:

```
0   long   1   372.21   372.21   372.21
```

— taxi 0 made exactly 1 long trip, fare $372.21, so max/min/avg are naturally identical.

5. `task1-run.sh`

```
#!/bin/bash
hadoop fs -rm -r -f /Output/task1

hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
  -D stream.num.map.output.key.fields=2 \
  -D mapred.reduce.tasks=3 \
  -file ./task1_mapper.py \
  -mapper ./task1_mapper.py \
  -file ./task1_reducer.py \
  -reducer ./task1_reducer.py \
  -input /Input/Trips.txt \
  -output /Output/task1 \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

Line by line

Line	Purpose
`hadoop fs -rm -r -f /Output/task1`	Hadoop refuses to write to an output directory that already exists. Deleting it first (silently, `-f` = no error if it doesn't exist) is what lets you run `./task1-run.sh` over and over with zero manual cleanup — required by "without any external or manual intervention."
`-D stream.num.map.output.key.fields=2`	Tells Hadoop Streaming that the first *two* tab-separated fields of each mapper output line (taxi_id, trip_type) together form "the key" for sorting/grouping purposes — not just the first field. Without this, Hadoop would only treat `taxi_id` as the key, and rows for the same taxi but different trip types wouldn't be distinguished during partitioning.
`-D mapred.reduce.tasks=3`	The explicit "exactly 3 reducers" requirement from the spec.

`-file / -mapper / -file / -reducer`	`-file` ships the script to every node in the cluster (so it's available wherever a mapper/reducer task happens to run); `-mapper`/`-reducer` say "run this executable as the mapper/reducer."
`-input / -output`	The required exact HDFS paths: `/Input/Trips.txt` in, `/Output/task1` out.
`-partitioner KeyFieldBasedPartitioner`	Hadoop's default partitioner would only look at the first field to decide which reducer a row goes to. Since we told Hadoop our key is 2 fields wide (via `stream.num.map.output.key.fields=2`), we also need a partitioner that's aware of multi-field keys — `KeyFieldBasedPartitioner` is the built-in Hadoop class for exactly that, and it's the same technique used in the Week 4 course material (`run_keys-1.sh`).

6. End-to-end example, traced through

Say taxi `104` has these three trips in `Trips.txt` (fare, distance only, for brevity):

```
fare=280.50, distance=150  → medium
fare=310.00, distance=175  → medium
fare=90.00,  distance=60    → short
```

Suppose the first two trips are read by mapper A and the third by mapper B (different input splits):

- **Mapper A's dict** ends with `("104","medium") → [2, 310.00, 280.50, 590.50]`, and emits:
  `104  medium  2  310.00  280.50  590.50`
- **Mapper B's dict** ends with `("104","short") → [1, 90.00, 90.00, 90.00]`, and emits:
  `104  short  1  90.00  90.00  90.00`
- Hadoop's shuffle/sort delivers both of these to whichever reducer owns key "104" — note they're *two different keys* (medium vs short), so the reducer treats them as separate groups, each getting their own `emit()` call:

```
104  medium  2  310.00  280.50  295.25
104  short  1  90.00  90.00  90.00
```

Two output rows for one taxi — exactly as the spec wants, since (taxi_id, trip_type) is the primary key, not taxi_id alone.

7. Requirement-by-requirement checklist

- ✅ Python + Hadoop Streaming only, no external MapReduce library.
- ✅ No full-dataset in-memory structure — the mapper's dict holds only per-key running aggregates (bounded by the number of distinct taxi/type pairs, not the number of trips), and the reducer holds only the single group currently being merged.
- ✅ In-mapper combining, state preserved across input lines (`stats` dict, declared outside the loop).
- ✅ Exactly 3 reducers (`mapred.reduce.tasks=3`, verified: 3 part-files produced on the real run).
- ✅ Exact HDFS paths: `/Input/Trips.txt`, `/Output/task1`.

- ✅ 6 tab-separated output fields, correct order, 2-decimal-place formatting on the fare fields, integer formatting on the count.
- ✅ `./task1-run.sh` runs end-to-end with no manual steps, and is safely re-runnable (self-cleans its output directory first).